

数据可视化

第四章 Python可视化库

上官武悦

2026年9月24日

本章安排

- Python可视化开发环境
- Python数据处理
  - NumPy：数值运算
  - Pandas：表格处理
- Python可视化库
  - 探索式可视化库：Matplotlib, Seaborn, Plotnine, Pyecharts,…
  - 交互式可视化库：Plotly, Bokeh, …

Python可视化开发环境

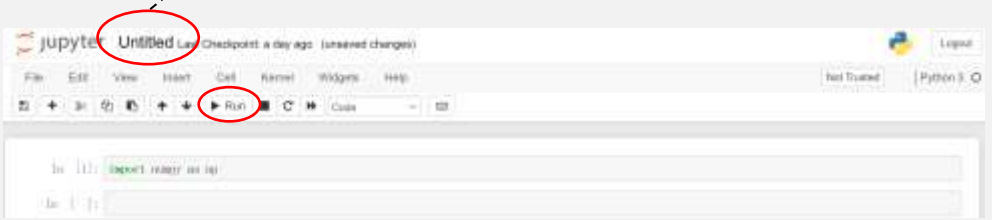


Jupyter Notebook

- Jupyter Notebook是一个在浏览器中使用的、基于Web的交互式笔记本，允许用户创建和共享包含实时代码、文本、图像和交互式工具的文档。其本质是一个Web应用程序，便于创建和共享程序文档，支持实时代码、数学方程、可视化和Markdown
- Jupyter Notebook的主要特点：
  - **交互式编程**：用户可以在Jupyter Notebook中逐个单元(cell)地编写和执行代码，并实时查看代码执行结果
  - **支持多种语言**：除了Python，Jupyter Notebook还支持多种其他编程语言，如R、Julia等
  - **富媒体展示**：用户可以在Jupyter Notebook中插入文本、图像、公式、交互式图表等，使得文档更加生动，分析结果更具表现力
  - **易于共享**：Notebook可以导出为HTML、PDF、Markdown等格式，方便用户分享分析结果

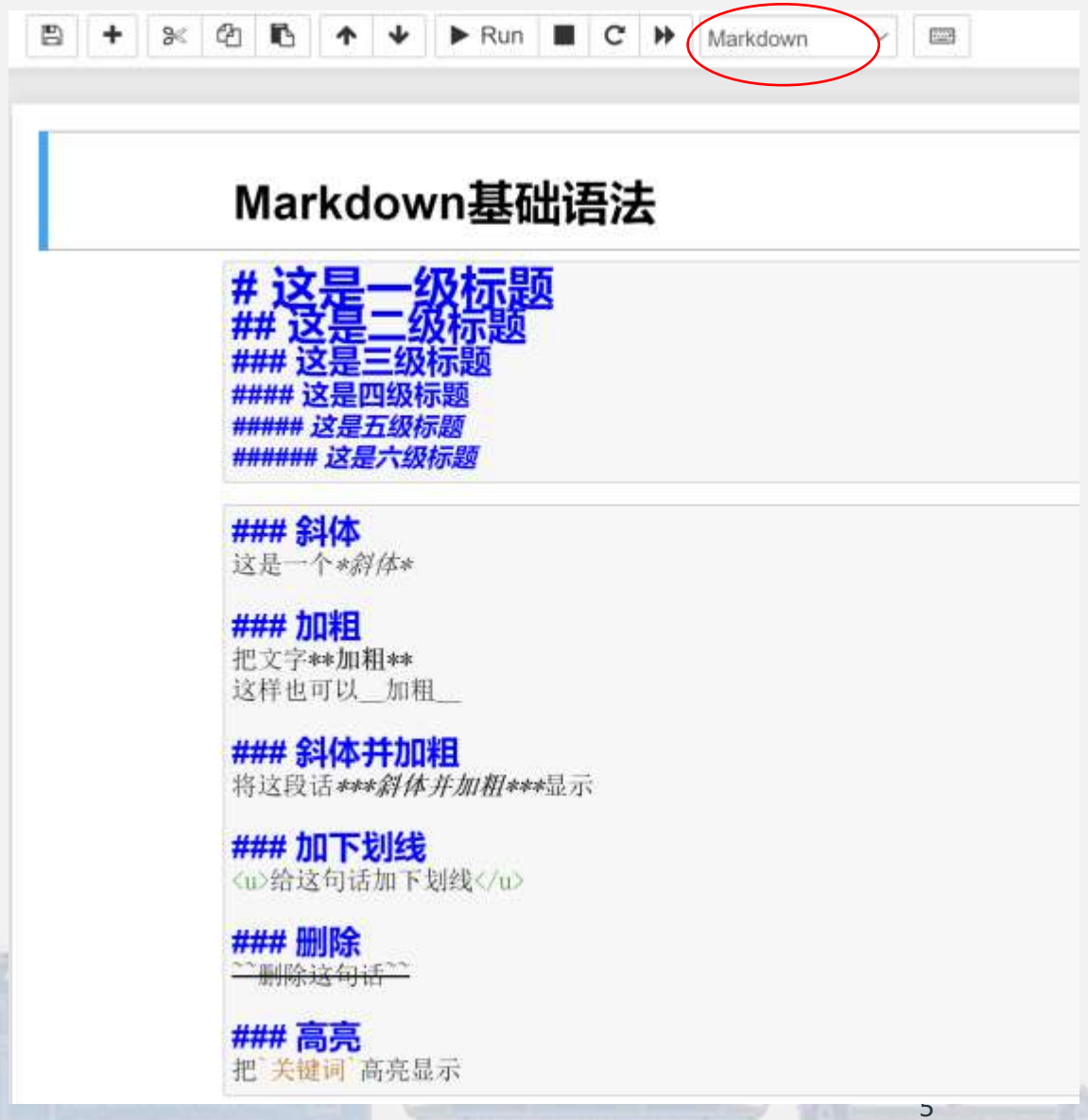


对当前的notebook进行重命名



Markdown

- 在Jupyter Notebook中，Markdown是一种常用的格式，用于编写描述性文本、注释、标题、列表、插入图片和超链接等，以使得Notebook的内容更加丰富和易读
- Markdown单元格在Jupyter Notebook中是以文本形式呈现的，但它们可以包含Markdown语法，这些语法在单元格被渲染时会转换成相应的格式
- 需要注意的是，在写markdown时，需要将单元格的类型选择成“Markdown”，而非“Code”
  - 按 Esc + M 切换到 Markdown 模式
  - 按 Esc + Y 切换到 Code 模式



课后作业

根据提供的Markdown基础语法.ipynb，学习和练习Markdown的语法

Python数据处理



NumPy



NumPy：数值运算

- NumPy是Numerical Python的简称，是高性能计算和数据分析的基础包
  - 官方文档： https://numpy.org/; 中文网站：https://www.numpy.org.cn/
- ndarray是NumPy的核心功能，含义为n-dimensional array，即多维数组
- 数组和列表(list)都可以进行切片和索引，但主要区别是：数组的所有元素必须具有相同的类型，而列表可以包含任意类型的元素
  - 使用.tolist()函数可以将数组转换为列表
- NumPy支持的数据类型有：
  - 布尔型(bool)、整型(int8, int16, int32, int64)、无符号整型(unit8, unit16, unit32, unit64)、浮点型(float16, float32, float 64)等，可以使用.astype()函数实现对数组数据类型的转换
- 如何查看NumPy是否已经安装及其版本?
  - pip show numpy

```
C:\Users\Phoebe>pip show numpy
Name: numpy
Version: 1.26.4
Summary: Fundamental package for array computing in Python
Home-page: https://numpy.org
```

- NumPy在使用前需要导入：
  - import numpy as np
  - print(np.__version__) #也可以查看numpy版本

NumPy：数值运算

- **数组**(ndarray)的概念
  - NumPy中，数组的维度是体现数组深度的一个属性，即数组有多少个轴（axis）
    - 例如，向量是一维数组，只有一个方向；一个二维数组可以看成由行和列组成的表格，第一维对应行，第二维对应列
  - NumPy中的轴用于指定在执行某些操作（如求和、均值等）时，沿哪个维度进行；axis=0表示第一维，axis=1表示第二维，以此类推
  - 将axis设为0时，轴是行，意味着操作将在列间执行，例如a.sum(axis=0)是求每列的和；将axis设为1时，轴是列，意味着操作将在行间执行，例如a.sum(axis=1)是求每行的和
  - 使用np.ndim()函数可查看数组的维度，使用np.shape()查看数组的形状

```
>>> import numpy as np
>>> a = np.array([[1,2,3],[2,3,4],[3,4,9]])
>>> a
array([[1, 2, 3],
       [2, 3, 4],
       [3, 4, 9]])
```

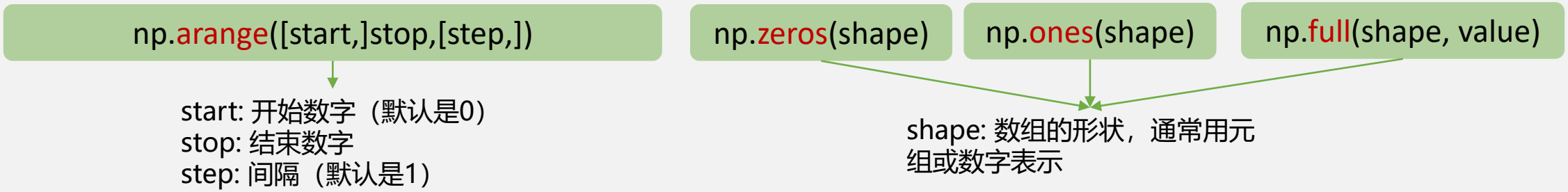
- np.ndim(a): 2
- np.shape(a): (3, 3)

NumPy：数值运算

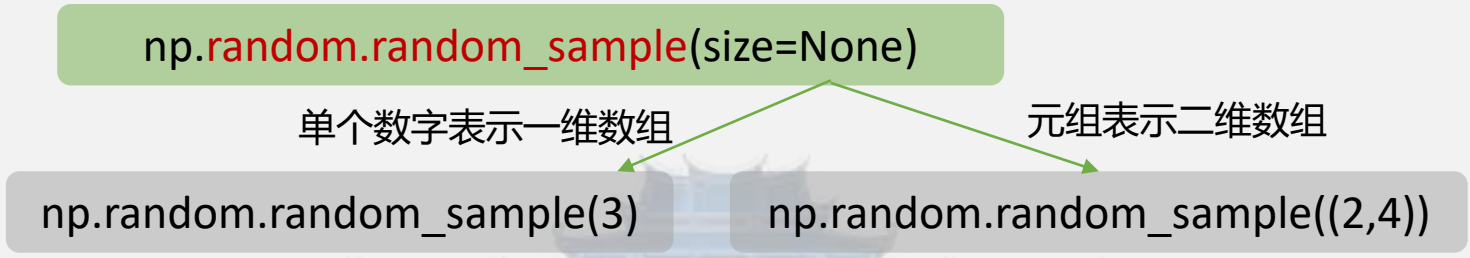
- 数组的**创建**方式
  - 从其他Python结构（例如列表、元组）转换



  - NumPy原生数组的创建（例如使用arange、ones、zeros等函数）



  - 使用特殊库函数（例如：random.random_sample）随机生成



NumPy：数值运算

- 数组的**索引与变换**：
  - 数组可以通过索引选取其中的元素、行或列。数组索引是指使用方括号（[]）来索引数组值
  - 可以根据指定条件进行过滤，选取符合条件的元素
  - 更改数组维度

    `np.reshape(a, shape)`

    `a.reshape(shape)`

    a: 要改变形状的数组
    shape: 指定形状，通常用元组或数字表示（注意：形状的改变应在元素数量不变的前提下，否则会报错）
  - 数组转置，实现行列互换

    `np.transpose(a)` → a: 想要转置的数组
  - 数组元素展开，将多维数组展开为一维数组

    `a.flatten()` → 返回原数组的拷贝，对拷贝所做的修改不会影响原数组

    `a.ravel()` → 返回视图，修改会影响原数组
  - 数组的排序

    `np.sort(a, axis=-1)`

    `a.sort(axis=-1)`

    a: 待排序数组
    axis: 默认取值为-1，沿最后一个轴排序
    取None会将多维数组展开成一维数组按升序排列

NumPy：数值运算

- 假设我们已执行如下代码：
  - import numpy as np
  - a = np.arange (6) # a = array([0,1,2,3,4,5])
- 数组的统计函数：

函数	功能	示例	结果
np.mean, np.average	计算平均值、加权平均值	np.mean(a)	2.5
np.var	计算方差	np.var(a)	2.917
np.std	计算标准差	np.std(a)	1.707
np.min, np.max	计算最小值、最大值	np.min(a), np.max(a)	0, 5
np.argmin, np.argmax	返回最小值、最大值对应的索引	np.argmin(a), np.argmax(a)	0, 5
np.ptp	计算值域，即最大值与最小值的差	np.ptp(a)	5
np.percentile	计算数组中的百分位数	np.percentile(a, 90)	4.5
np.median	计算中位数	np.median(a)	2.5
np.sum	计算数组中所有元素之和	np.sum(a)	15

课堂练习

1. 创建一个3x4维的二维数组，其中所有元素都为0，并打印该数组
2. 创建一个包含从0到9所有整数的一维数组，然后将其重塑（reshape）为一个2x5的二维数组
3. 将以下2x3的数组变换为一个3x2的数组

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
```

4. 计算上一题中的二维数组arr每一列的平均值
5. 计算上一题中的二维数组arr每一行的标准差

Pandas



Pandas：表格处理

- Pandas 是 Python 的一个扩展程序库，在分析结构化数据方面非常强大，广泛应用在管理学、统计学、经济学等各个数据分析领域
  - 官方文档：https://pandas.pydata.org/docs/；中文网站：https://pandas.ac.cn/docs/
- Pandas 名字衍生自术语 “panel data”（面板数据）和 “Python data analysis”（Python 数据分析）
- Pandas 可以对数据进行多种操作和运算，比如读取、清洗、过滤、统计、分析、汇总、透视、特征工程等
- Pandas 可以从各种文件格式比如TXT、CSV、JSON、SQL、Microsoft Excel 导入数据
- 初次使用Pandas，需要先安装【查看安装方式】
- 每次使用Pandas之前，需要先导入
  - import pandas as pd

Pandas：表格处理

- Pandas 的主要数据结构是 Series （一维数据）与 DataFrame（二维数据）
  - Series 是一个一维的数组结构，它可以存储任何数据类型（如日期、整型、浮点数、字符串等），通常由一组数据以及与之相关的索引（index）组成
    - 默认情况下，索引是从0开始的整数序列，但也可以更改成其它类型的值（如日期、字符串）
  - DataFrame 是一个二维的表格结构，它可以存储任何数据类型（如日期、整型、浮点数、字符串等），类似于Excel 表格。一个DataFrame包含若干列，因此DataFrame既有行索引也有列索引
    - 同样，索引默认从0开始，也可以手动更改行索引为其他类型（如日期、字符串）

Pandas：表格处理

- 创建Series对象，查看索引和取值
  - import pandas as pd
  - s1 = pd.Series([1,3,5,7,9])
  - 查看索引：s1.index（返回一个index对象：RangeIndex(start=0, stop=5, step=1)）
  - 查看取值：s1.values（返回一个array对象：array([1, 3, 5, 7, 9], dtype=int64)）
  - 查看第1个位置的元素取值：s1.iloc[0]、s1[0]

  - s2 = pd.Series([1,3,5,7,9], index=['a', 'b', 'c', 'd', 'e'])
  - s2.index（返回一个index对象：Index(['a', 'b', 'c', 'd', 'e'], dtype='object')）
  - s2.values（返回一个array对象：array([1, 3, 5, 7, 9], dtype=int64)）

Pandas：表格处理

➢创建DataFrame对象

1. 从列表(list)创建

```
df = pd.DataFrame([['a',1,2], ['b',2,5], ['c',3,3]], columns=['x', 'y', 'z'])
```

x	y	z
a	1	2
b	2	5
c	3	3

2. 从多维数组(ndarray)创建

```
df = pd.DataFrame(np.zeros((3,3)), columns=['x', 'y', 'z'])
```

x	y	z
0	0	0
0	0	0
0	0	0

3. 从字典(dictionary)创建

```
df = pd.DataFrame({'x':['a','b','c'], 'y':range(1,4), 'z':[2,5,3]})
df = pd.DataFrame(dict(x=['a','b','c'], y=range(1,4), z=[2,5,3]))
```

x	y	z
a	1	2
b	2	5
c	3	3

Pandas：表格处理

➢创建DataFrame对象

4. 从文件创建

读取CSV文件：pd.read_csv(*filepath*, sep=',')

读取TXT文档：pd.read_table(*filepath*, sep='\t')

读取Excel文件：pd.read_excel(*filepath*, sheet_name=0, header=0)

读取JSON文件：pd.read_json(*filepath*)

读取Stata数据(.dta)：pd.read_stata(*filepath*)

Pandas：表格处理

- DataFrame数据选取
  - 熟悉loc、iloc函数的使用。loc[]通常根据行索引和列索引来选取，iloc[]根据索引下标位置选取

df

x	y	z
a	1	2
b	2	5
c	3	3

- 选取某一列：df[['y']]、df.iloc[:,[1]] 、df.loc[:,['y']] 、df['y']、df.y、df.loc[:, 'y']

数据类型仍是DataFrame　　数据类型变为Series

- 选取多列：df[['x','y']]、df.iloc[:,[0,1]] 、df.loc[:,['x', 'y']]
- 选取某一行：df.loc[0,]、 df.iloc[0,]
- 选取多行： df.loc[1:,]、 df.iloc[1:,]
- 选取某个元素：df.loc[1, 'y']、 df.iloc[1,1]
- 选取符合条件的数据：df[df.z>=3]、 df.loc[df.z>=3]、 df[(df.z>=3) & (df.y<3)]

Pandas：表格处理

- 查看DataFrame的属性信息：df.info()
- 查看DataFrame前五行的数据：df.head()
- 查看DataFrame后五行的数据：df.tail()
- 查看DataFrame的描述性统计：df.describe()
- 获取DataFrame的行数：len(df)、df.shape[0]
- 获取DataFrame的列数： df.shape[1]
- 查看DataFrame的行索引：df.index
- 查看DataFrame的列索引（列名）：df.columns
- 重新定义所有列的列名：df.columns=['X', 'Y', 'Z']
- 更改某一列的列名：df.rename(columns={'x':'X'}, inplace=True)

Pandas：表格处理

➢缺失值处理

- ➢ Python中的缺失值有三种情况：np.nan、pd.NaT（时间格式的空值）以及Python内置的None值
- ➢ 注意，空值(np.nan, pd.NaT, None)既不是空字符串""，也不是空格" "

➢判断是否存在缺失值

df.isna()
df.isnull()
} 缺失的位置显示True，否则False

df.notnull()
df.notna()
} 与前者相反

➢删除

df.dropna(axis=0, how='any', threshold, subset, inplace=False)

- axis=0: 删除有缺失值的行
  axis=1: 删除有缺失值的列
- any: 存在空值就删除
  all: 全部为空值才删除
- 指定阈值，非空值超过threshold才删除
- 在指定的子集中考虑删除；若是按行删除，这里可以指定列的子集

➢填充

df.fillna(value=None, method=None, axis=None, inplace=False)

- 用什么值来填充缺失值，可以用dict指定每列要填充的值
- 填充方法
  ffill：缺失值前面的值代替
  bfill/backfill：缺失值后面的值代替
- 按行填充还是按列填充
  对于Series，默认按行填充，即axis=0

Pandas：表格处理

- 表格排序：

  按列名排序：df.sort_values(by='z', ascending=True, inplace=False)

  按索引排序：df.sort_index(ascending=True, inplace=False)

- 表格拼接：

  按列拼接：pd.concat([df1,df2], axis=1)

  按行拼接：pd.concat([df1,df2], axis=0)

x	y	z
a	1	2
b	2	5
c	3	3

df



x	y	z
a	1	2
c	3	3
b	2	5

按列z升序排列

x	y	z	x	y	z
a	1	2	0	0	0
b	2	5	0	0	0
c	3	3	0	0	0

df1　　df2

x	y	z
a	1	2
b	2	5
c	3	3
0	0	0
0	0	0
0	0	0

Pandas：表格处理

➢合并：

df1.merge(df2, left_on='x', right_on='x', how='inner')

df1.merge(df2, on='x')

df1.merge(df2, on='x', how='left')

df1.merge(df2, on='x', how='right')

x	y	z
a	1	2
b	2	5
c	3	3

+

x	w	v
a	1	6
b	2	7
c	3	8
d	4	9
e	5	10

内连接

x	y	z	w	v
a	1	2	1	6
b	2	5	2	7
c	3	3	3	8

右连接

x	y	z	w	v
a	1	2	1	6
b	2	5	2	7
c	3	3	3	8
d			4	9
e			5	10

➢分组：

按照列值进行分组：df.groupby(by='x')

返回groupby对象，包含分组信息，既有分组依据的键，也有每个组的数据

➢描述统计：

df.y.mean()、df.y.sum()、df.y.std()、 df.y.max()、 df.y.min()、 df.y.median()、 df.y.mode()……

Pandas：表格处理

➢DataFrame导出到文件

导出CSV文件：df.to_csv(*filepath*, index=False)

导出Excel文件：df.to_excel(*filepath*, sheet_name='Sheet1', index=False)

导出JSON文件：df.to_json(*filepath*)

导出Stata文件： df.to_stata(*filepath*)

课堂练习

➢根据提供的数据集：customers.txt

1) 读取数据到DataFrame中
2) 查看数据集中各个属性（列）的类型
3) 打印前五行和后五行的数据
4) 查看属性education、occupation、income的可能取值有哪些
5) 选择出客户ID为"Cust-21910"的行
6) 将marital字段重新赋值：是改为1，否改为0
7) 将数据拆分为男性客户和女性客户两个样本
8) 依次输出男性客户中，高价值客户、中等价值客户、低价值客户的教育水平的众数、已婚人数比例，以及年龄的均值
9) 将女性客户样本导出到Excel中

Python可视化库



Matplotlib

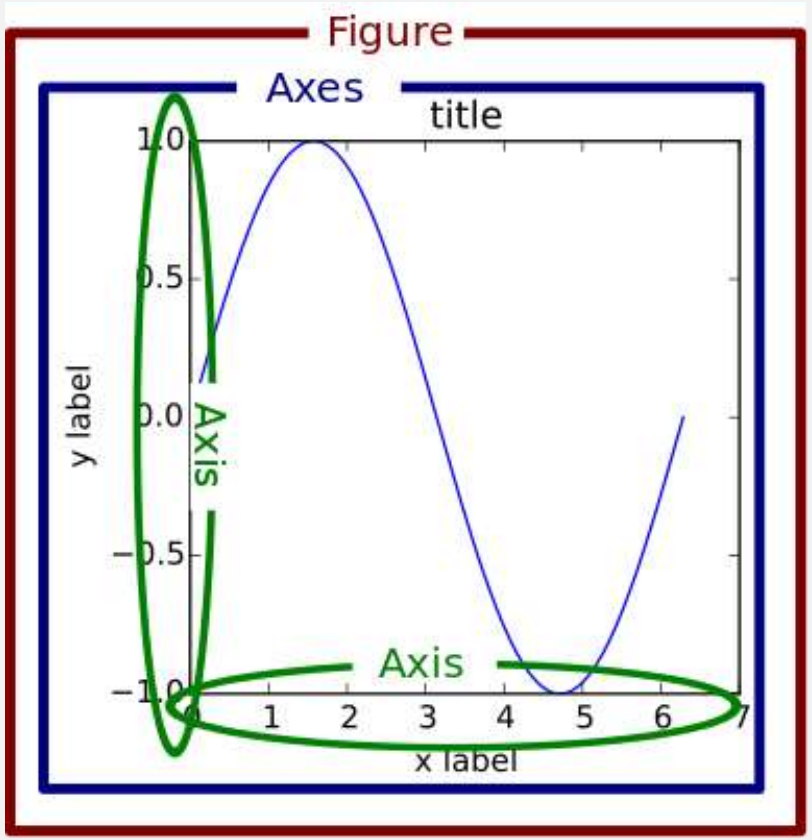


Matplotlib简介

- **Matplotlib**是一个基础的Python可视化库，它基于NumPy的数组运算功能，可以绘制静态、动态、交互式图表和图形，功能非常强大，广泛应用在数据科学领域
  - 官方网站：https://matplotlib.org/；中文网站：https://www.matplotlib.net/
- Matplotlib的作用和特点：
  - **绘制多种图形**：Matplotlib 能够绘制线图、散点图、条形图、直方图、饼图、堆叠图、面积图等多种图形，满足不同场景的数据可视化需求
  - **定制化强**：用户可以调整图形的几乎每一个元素，包括线型、字体、颜色、风格、坐标轴等，以创建高度个性化的图表
  - **输出格式多样**：Matplotlib 支持多种图像输出格式，包括 PNG、PDF、SVG（可缩放矢量图）、EPS（封装的PostScript格式），便于在不同媒介上使用
  - **集成性强**：可以很容易地将 Matplotlib 集成到 Python 的科学计算环境中，与 NumPy、SciPy、Pandas 等数据分析库协同工作

Matplotlib简介

- Matplotlib主要包含两类元素：
  - **基础类**（primitives）：线(line)、点(marker)、文字(text)、图例(legend)、网格(grid)、标题(title)、图片(image)等 → 要绘制的标准对象
  - **容器类**（containers）：图形(figure)、坐标系/子图(axes)、坐标轴(axis)、刻度(tick) → 包含基础类元素并将它们组织成一个整体；存在层级结构



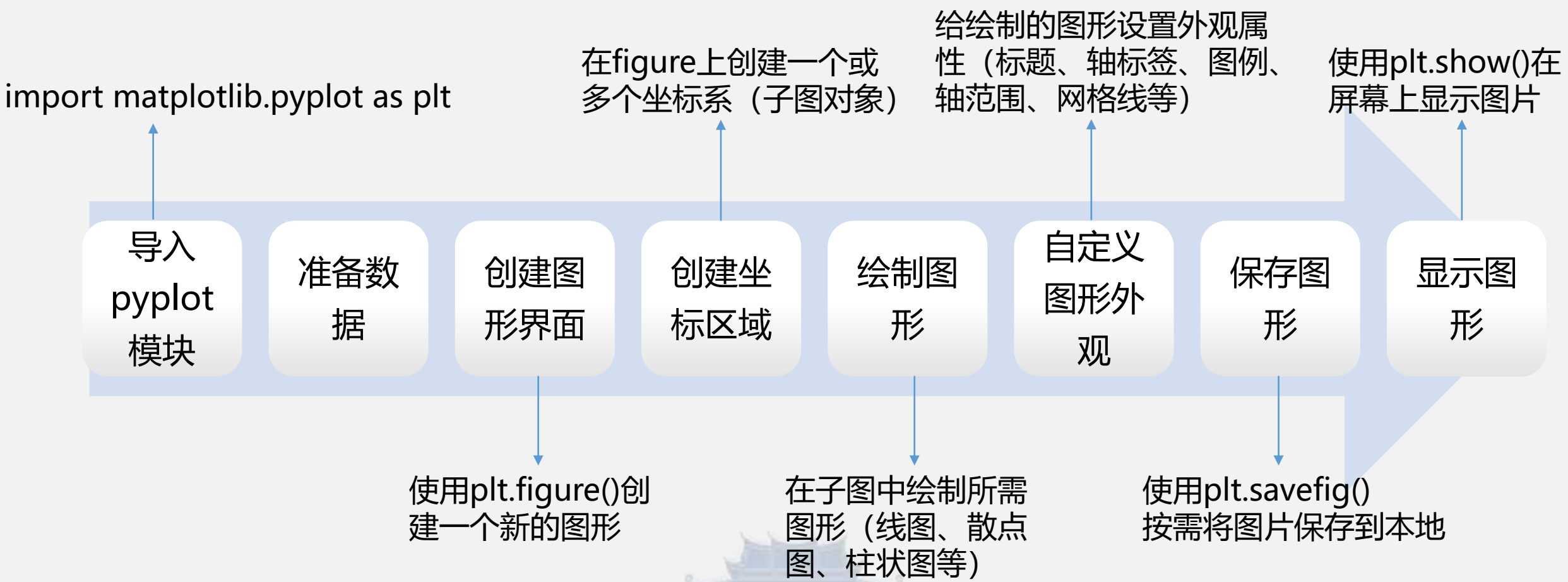
Matplotlib容器类元素

- **图形/figure 对象**：整个图形界面即一个figure对象，至少包含一个子图(即axes对象)
  - figure对象包含canvas对象，但是canvas对象一般不可见，通常无须直接操作，但是matplotlib在实际绘图时需要调用该对象
- **子图/axes 对象**：虽然axes是axis的复数，但它并不是指坐标轴，而是子图对象，也可以理解为坐标系、坐标区域。尤其是在同一个图形界面绘制多个子图时，需要使用axes对象
  - 每个子图都有横轴和纵轴，axes用于代表这两个坐标轴所对应的子图对象
  - set_xlim() & set_ylim(): 设置子图X轴和Y轴对应的数据范围
  - set_xlabel() & set_ylabel(): 设置子图X轴和Y轴的标签名称
  - set_title(): 设置子图的名称
- **坐标轴/axis 对象**：坐标轴（数据轴）对象，主要用于控制数据轴上的刻度位置和数值
  - axis有locator和formatter两个子对象，分别用于控制刻度位置和显示数值
- **刻度/tick 对象**：每个坐标轴包含刻度元素，刻度元素包含刻度本身和刻度标签

Matplotlib绘图接口(API)

- **Pyplot API**：Matplotlib的接口之一，设计灵感来自于 MATLAB。每个pyplot函数都会对当前图形进行一些改动，比如创建一个图形、在图形中创建绘图区域、在绘图区域中绘制一些线，使用标签和图例装饰图形等
  - import matplotlib.pyplot as plt *#约定俗称的调用形式*
  - plt.figure() *#创建一个图形对象*
  - plt.subplot() *#在当前图形上添加子图*
- **面向对象的 API**: 面向对象的 API 更复杂，但也提供了更好的控制和定制性。面向对象的API会直接与图形对象、子图对象和其他更细粒度的元素交互。使用面向对象的 API需要先创建图形（Figure）和子图（Axes）对象，然后通过这些对象的方法来构建图形
  - fig, ax = plt.subplots() *#创建一个包含子图的图形并返回对子图的引用*

Matplotlib绘图流程



Matplotlib图表绘制

- 使用前需导入：**import matplotlib.pyplot as plt**
- 创建并返回一个图形对象 (figure)：

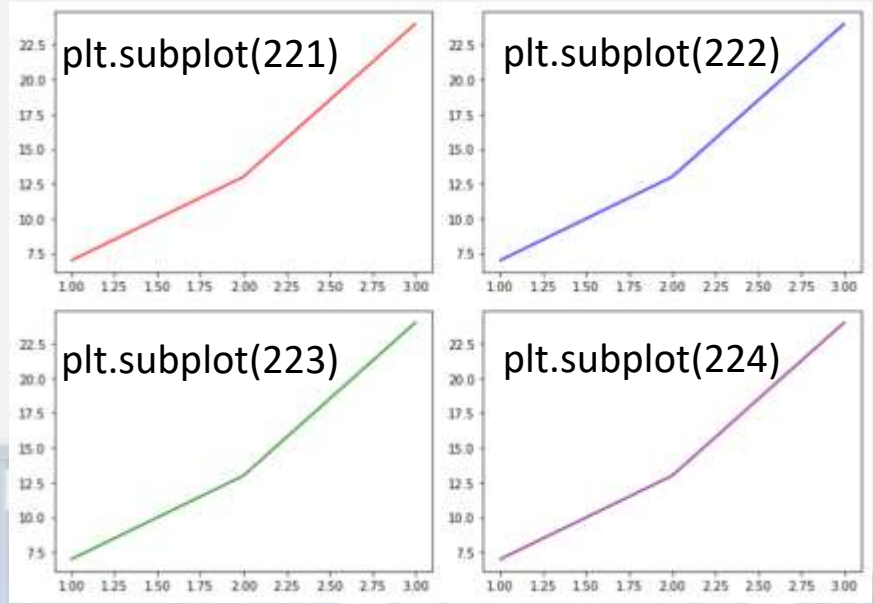
```
plt.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True)
```

- 唯一标识符. 如果有该标识符的图形已经存在，则该图形将被激活并返回；否则会创建一个新数字，处于激活状态并返回
- 图形大小（宽，高）：格式是(float, float)，默认值是(6.4,4.8)
- 分辨率：float类型，默认值100
- 背景色：默认值'white'
- 边框颜色：默认值'white'
- 是否绘制图形边框：默认值是True

- 使用subplot()逐个创建子图 (axes)：

```
plt.subplot(nrows, ncols, index, *args, **kwargs)
```

- 将整个绘图区域分成nrows行和ncols列，乘积为图形总共被划分的子图区域个数，编号从1开始，从左到右逐行增加
- index为当前正在绘制的子图的区域编号
- 参数中间的逗号可以省略
- 每次创建一个编号为index的子图



Matplotlib图表绘制

➢使用subplots()一次性创建图形和所有子图

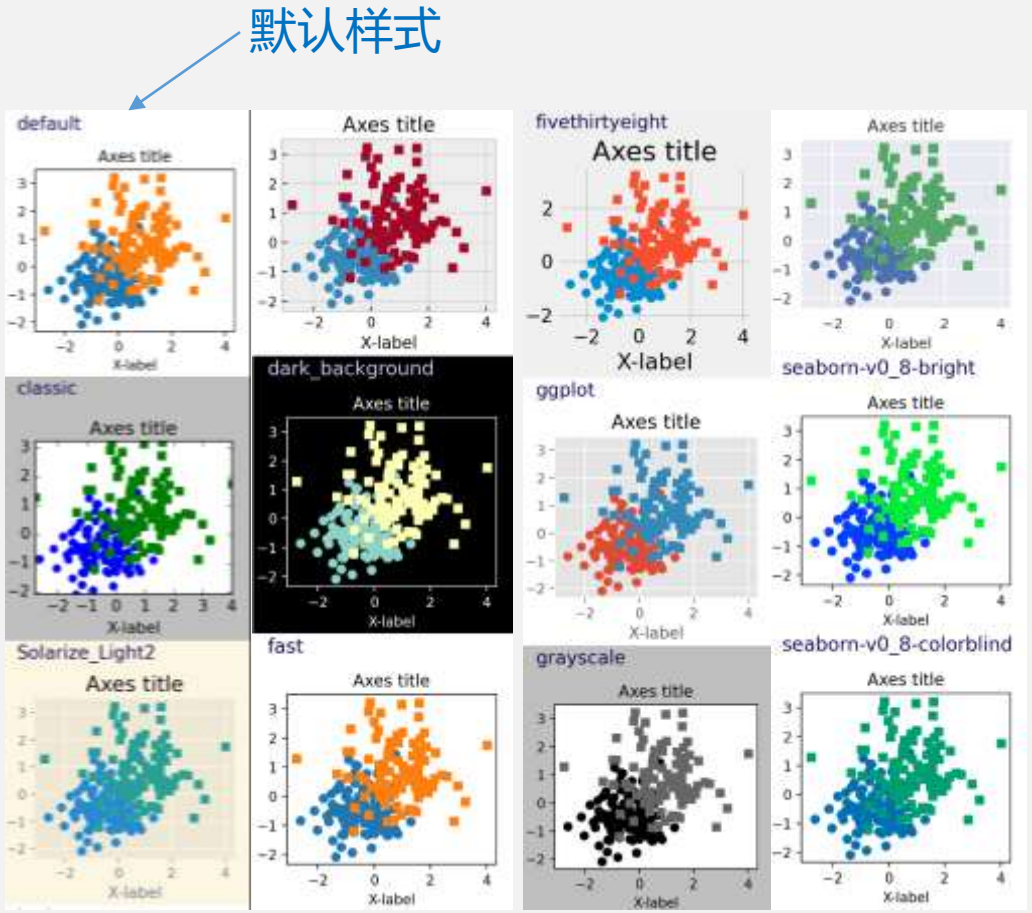
```
fig, ax = plt.subplots(nrows=1, ncols=1, sharex=False, sharey=False,**kwargs)
```

- 图形被分割成的行数，默认是1
- 图形被分割成的列数，默认是1
- 子图是否共用横坐标和纵坐标
  - True or 'all': 所有子图共用;
  - False or 'none': 每个子图的坐标是独立的，不共用
  - 'row': 每一行子图共享
  - 'col': 每一列子图共享

- fig, ax = plt.subplots() *# 图形中仅包含一个子图对象*
- fig, axs = plt.subplots(2, 2) *# axs是一组子图对象的二维数组，axs[0][0]~axs[1][1]，也可以写成axs[0,0] ~axs[1,1]*
- fig, (ax1, ax2) = plt.subplots(1, 2) *# 使用元组分别表示两个子图*
- fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2) *# 使用元组分别表示四个子图*

Matplotlib图表样式

- 查看可用的图表样式：plt.style.available
- 使用某个样式： plt.style.use('ggplot')
  - 需要注意的是，设置图表样式之后，此后就会一直使用该样式，直到再次更改样式或重置为默认值
  - 重置为默认样式： plt.style.use('default')
- 查看某个样式的具体属性设置：
  - plt.style.library['seaborn-darkgrid']



Matplotlib绘图操作

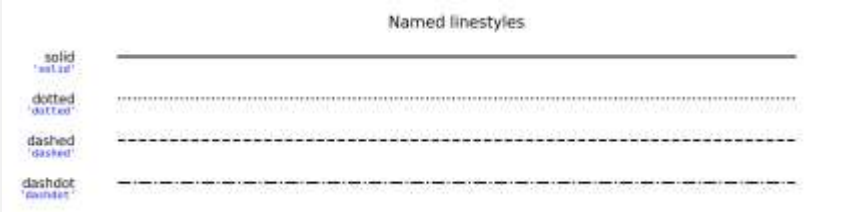
- 调整Matplotlib绘图属性需要对图表元素进行操作
  - 返回当前状态下的axes：plt.gca()
  - 查看当前axes对象下的元素：plt.gca().get_children()
  - 设置图形居中的标题：plt.suptitle()
  - 设置当前子图的标题：plt.title()
  - 设置X轴和Y轴的标题：plt.xlabel(), plt.ylabel()
  - 设置X轴和Y轴的取值范围：plt.xlim(), plt.ylim()
  - 设置X轴和Y轴的轴刻度： plt.xticks(), plt.yticks()
  - 设置网格线：plt.grid()
  - 设置图例：plt.legend()

Matplotlib图表修饰属性

- 标记 (marker)

```
filled_markers = ('.', 'o', 'v', '^', '<', '>', '8', 's', 'p', '*', 'h',
'H', 'D', 'd', 'P', 'X')
```

- 线型 (linestyle)：可以使用字符串'solid' (or '-')、'dotted' (or ':') 、'dashed' (or '--')或'dashdot' (or '-.')来定义简单的线型



- 字体 (font family)：通过设置 plt.rcParams['font.family']，告诉Matplotlib使用选择的字体来渲染文本，可以使用通用的字体样式或在电脑上安装其他字体样式
  - 默认字体是：rcParams['font.family'] = 'sans-serif'
  - 常见字体有：‘serif’ (衬线字体，如Times New Roman或Georgia，常用于打印和文档), ‘sans-serif’(无衬线字体，如Arial或Helvetica，常用于网页和屏幕显示), ‘monospace’ (等宽字体), ‘fantasy’ (艺术字体) or ‘cursive’(手写字体)
  - 查看电脑上已安装的字体：C:\Windows\Fonts



Matplotlib图表修饰属性

- 颜色 (color)：Matplotlib中的颜色有多种设置方式
  - 使用浮点值的RGB (Red, Green, Blue) 或RGBA元组，其中A表示Alpha（即透明度，0表示完全透明，1表示完全不透明)
    - 例如(0.1, 0.2, 0.5)或(0.1, 0.2, 0.5, 0.3)
  - 使用十六进制RGB或RGBA字符串，例如'#0F0F0F'或'#0F0F0F0F'
  - 使用单字母字符串，例如'b'(blue)、'g'(green)、'r'(red)、'c'(cyan)、'm'(magenta)、'y'(yellow)、'k'(black)、 'w'(white)等
  - 使用X11/CSS4颜色名称，例如'blue'、'grey'、'aquamarine'、'mediumseagreen'等
    - # 查看所有颜色名称
    - import matplotlib.colors as mcolors
    - print(mcolors.CSS4_COLORS.keys())

Matplotlib图表绘制

➢ matplotlib.pyplot基础图表函数

函数	核心参数说明	图表类型
plot()	x, y, color(线条颜色), linestyle (线型), linewidth (线条宽度), marker (标记类型), markeredgecolor (标记边框颜色), markeredgewidth (标记边框宽度), markerfacecolor (标记填充颜色), markersize (标记大小), label (线条标签)	线图、带数据标记的线图
scatter()	x, y, s (散点大小), c (散点颜色)	散点图、气泡图
bar()	x, height (高度), width (宽度), align (位置), color (颜色)	柱状图、堆积柱状图
barh()	x, width (宽度), height (高度), align (位置), color (颜色)	水平条形图、水平堆积条形图
hist()	x, bins (分箱总数), range (统计范围), density (是否绘制概率密度), cumulative (是否绘制累积分布), histtype (直方形状), align (位置), color (颜色), orientation (水平还是竖直)	直方图、密度分布图
boxplot()	x, notch (有无凹槽), sym (异常值的符号), vert (竖直或水平方向), whis (箱须长度), widths (箱型宽度)	箱型图
violinplot()	x, positions (位置), vert (竖直或水平方向), showmeans (是否用短线显示均值), showmedians (是否用短线显示中位数), bwmethod (计算带宽的方式)	小提琴图
pie()	x, colors (颜色), labels (标签)	饼图
errorbar()	x, y, yerr (Y轴方向误差范围), xerr (X轴方向误差范围), fmt (数据点的标记和连接样式), ecolor (误差棒颜色), elinewidth (误差棒宽度), ms (数据点大小), mfc (数据点标记填充颜色), mec (数据点标记边缘颜色), capthick (误差棒横杠的粗细), capsize (误差棒横杠的大小)	误差棒图

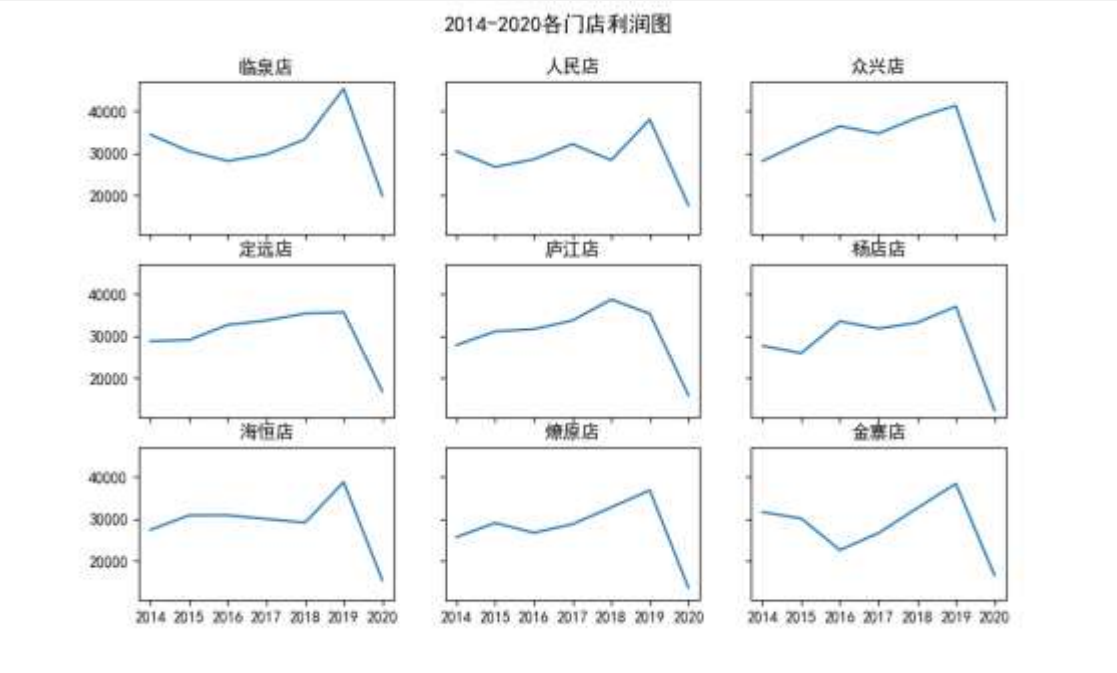
Matplotlib图表导出

- Matplotlib图表可以导出不同的格式，包括pdf、png、jpg、svg (可缩放矢量图形)等
- 使用plt.savefig()进行图表导出
- 例如，导出pdf格式的图表
  - plt.savefig('filename.pdf', format='pdf')
- 需要注意的是，要在plt.show()之前调用plt.savefig()，否则就会保存一个新生成的空白图片
- 在其他一些情况下，也会出现导出空白图片的情况，这时可以增加一些参数设置，使图片可以显示出来
  - 例如，设置plt.savefig('filename.png', format='png', bbox_inches='tight')

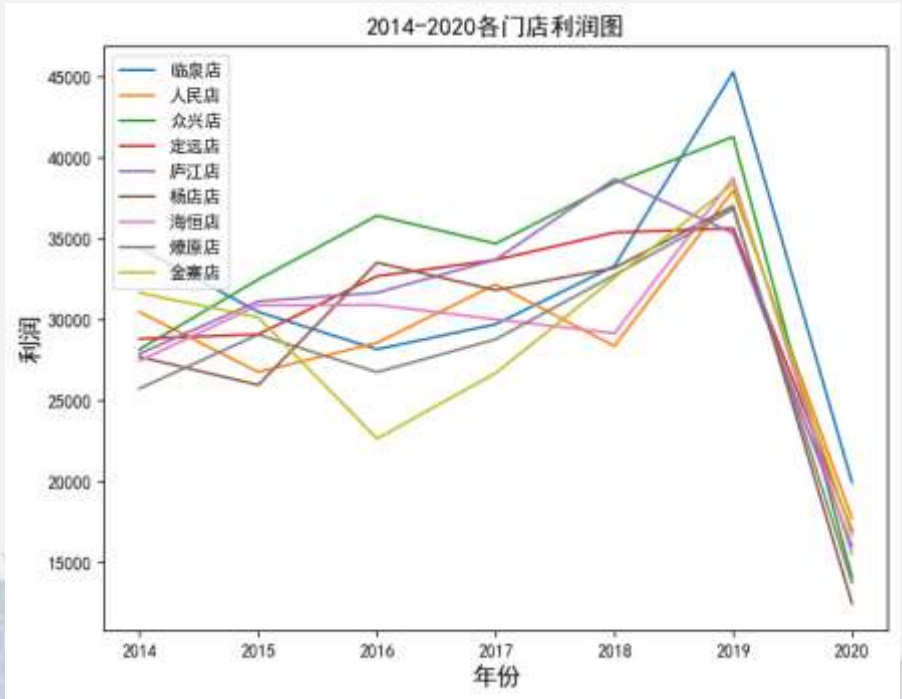
边界框设置为紧

课堂练习

- 根据提供的数据集orders.txt
  - 统计各个门店每个年份的利润总和
  - 绘制2020年各个门店利润的条形图，导出为png图片
  - 绘制各个门店2014-2020期间的利润折线图，导出为pdf文件



或者



Seaborn



Seaborn

- Seaborn是一个基于Matplotlib的Python可视化库，它在Matplotlib的基础上进行了更高级的API封装。这也意味着如果需要对图形进行更深层次的自定义，可以直接使用Matplotlib的功能
- Seaborn提供了一系列高级的统计图形绘制函数，同时它也支持NumPy和Pandas等Python科学计算的核心库，使得创建有吸引力的统计图形变得更加直观和容易
- Seaborn官方网站: https://seaborn.pydata.org/，中文网站：https://seaborn.org.cn/
- 安装Seaborn需要Python版本3.8+，且需要先安装NumPy、Pandas、Matplotlib
- 使用Seaborn之前需要导入
  - import seaborn as sns

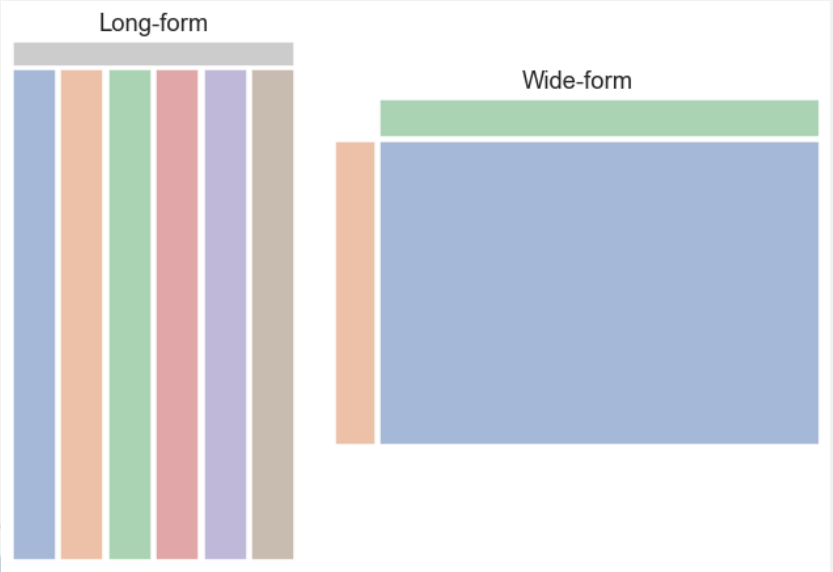


Seaborn vs. Matplotlib

- **抽象层次：**
  - Matplotlib是一个较底层的库，提供了大量的函数来定制图表的每一个细节。它是一个非常强大且灵活的工具，但这也意味着用户在创建复杂图表时通常需要编写大量的代码
  - Seaborn提供了更高层次的抽象，专注于提供简单易用的接口来创建常见的统计图表。Seaborn的API旨在减少绘图所需的代码量，并且使得图表的默认样式更加吸引人
- **样式和美观：**
  - Matplotlib默认的样式较为简单，用户通常需要额外的代码来美化图表
  - Seaborn具有多种预设的美观样式和颜色主题，可以轻松地创建外观更加时尚美观的图表
- **数据集成：**
  - Matplotlib对于数据的输入不是特别挑剔，通常接受基本的Python数据结构如列表, NumPy数组等
  - Seaborn更好地整合了Pandas，可以直接使用Pandas的DataFrame作为数据源，并且可以利用DataFrame中的列名来指定变量，使得数据操作更加方便
- **统计功能：**
  - Matplotlib主要关注于绘图，不直接提供统计功能。如果需要统计分析，用户通常需要结合如NumPy, SciPy或Pandas等库来计算统计数据
  - Seaborn内置了一些统计功能，可以直接绘制带有统计分析的图表，例如回归图等
- **图表类型：**
  - Matplotlib提供了非常基础的图表类型，用户可以通过组合这些基础的组件来创建复杂的图表
  - Seaborn提供了更多专门为统计分析设计的图表类型，图形的分面显示更容易

Seaborn接受的数据结构

- Seaborn支持几种不同的数据集格式，大多数函数接受Pandas或NumPy的数据对象以及内置Python类型（如列表和字典）表示的数据
- Seaborn中大多数绘图函数都针对数据向量，也就是说每个变量都应该是一个向量。Seaborn接受具有以某种表格方式组织的多个向量的数据集（例如DataFrame）
- Seaborn内置了一些示例数据集：sns. get_dataset_names()



- 长格式数据 (Long-form data)：每个变量是一列、每个观测值是一行
- 宽格式数据 (Wide-form data)：每个对象的多个观测值被存储在同一行中，并且每个观测值都有自己的列

Seaborn接受的数据结构

- 长格式 vs. 宽格式

```
flights = sns.load_dataset("flights")
flights
```

	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121
...	...	...	...
139	1960	Aug	606
140	1960	Sep	508
141	1960	Oct	461
142	1960	Nov	390
143	1960	Dec	432

将长格式转换为宽格式：
Pandas.DataFrame.pivot()

将宽格式转换为长格式：
Pandas.DataFrame.melt()

```
flights_wide = flights.pivot(index="year", columns="month", values="passengers")
flights_wide.head()
```

month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
year												
1949	112	118	132	129	121	135	148	148	136	119	104	118
1950	115	126	141	135	125	149	170	170	158	133	114	140
1951	145	150	178	163	172	178	199	199	184	162	146	166
1952	171	180	193	181	183	218	230	242	209	191	172	194
1953	196	196	236	235	229	243	264	272	237	211	180	201

```
# 宽格式转长格式数据
flights_wide.melt(value_vars=flights_wide.columns.values,value_name='passsengers',ignore_index=False)
```

year	month	passsengers
1949	Jan	112
1950	Jan	115
1951	Jan	145
1952	Jan	171
1953	Jan	196
...	...	...
1956	Dec	306
1957	Dec	336
1958	Dec	337
1959	Dec	405
1960	Dec	432

144 rows × 2 columns

Seaborn绘图设置

- Seaborn的优势之一是可以快速设定图表风格和颜色主题
- sns.set_theme(): 使用默认主题‘darkgrid’ (灰色网格)
- sns.set_style(): 设置绘图的总体风格 [查看API]
  - Seaborn内置的图表风格有: ‘darkgrid’ (默认设置), ‘whitegrid’, ‘dark’, ‘white’, ‘ticks’
- sns.set_context(context, font_scale=1, rc): 设置绘图元素的缩放比例，如标签大小、线宽等
  - context可以设置为内置类型: ‘paper’, ‘notebook’ (默认设置), ‘talk’, ‘poster’
  - font_scale即缩放因子，用于独立缩放字体元素的大小
  - rc是字典形式的参数映射
- sns.set_palette(palette): 设置绘图的颜色主题 [查看API]
  - palette应指定Seaborn颜色调色板: 定性调色板（表示类别数据）、顺序调色板（表示连续数据）、发散调色板（表示具有分类边界的类别数据）

Seaborn绘图函数

- Seaborn绘图函数是面向数据集的，与DataFrame进行了很好的集成
- Seaborn统计制图的类型：

关系图：
relplot()

- 散点图 scatterplot()
- 线图 lineplot()

分布图：
displot()

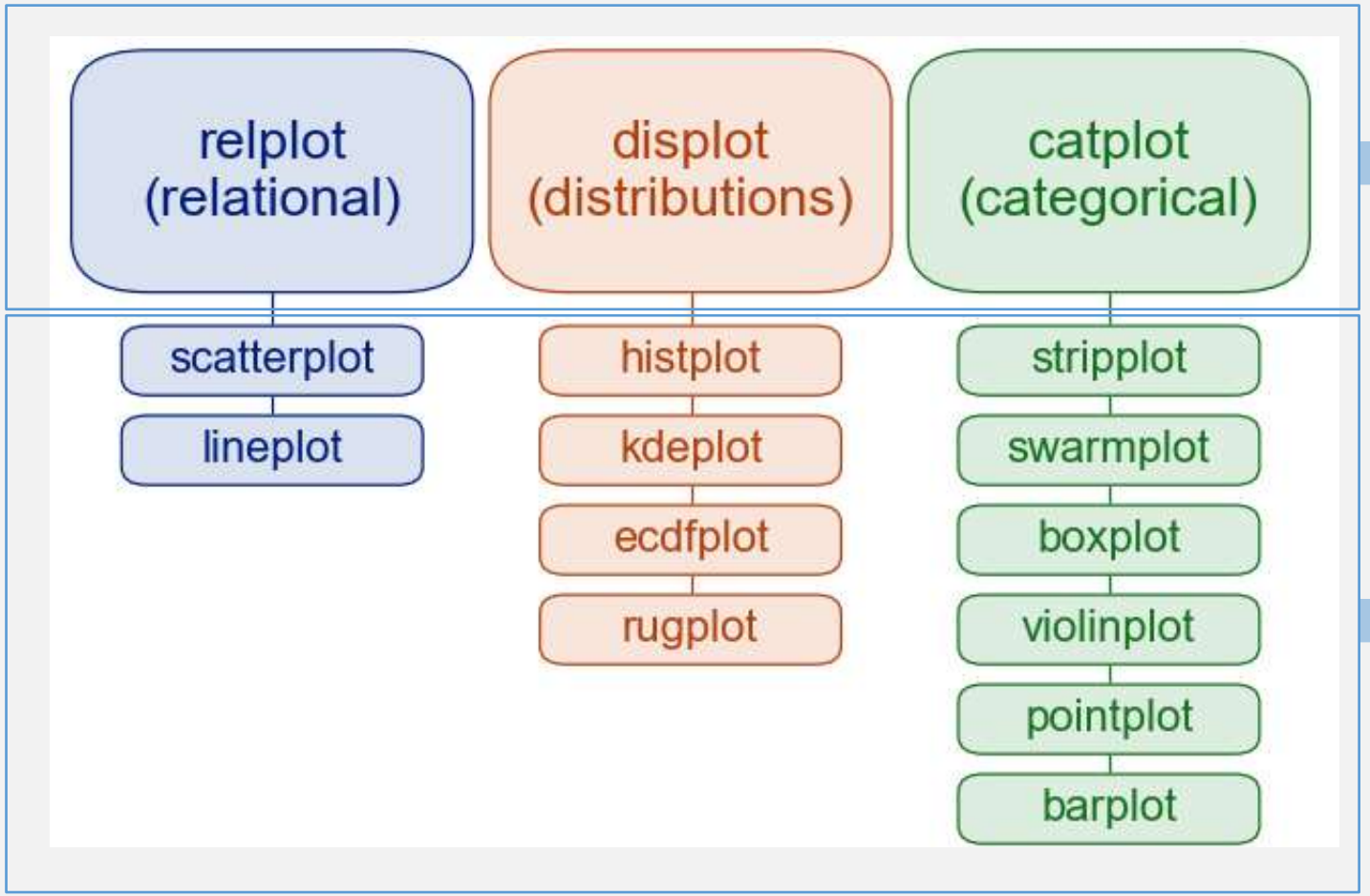
- 直方图 histplot()
- 核密度图 kdeplot()
- 经验累积分布图 ecdfplot()

分类图：
catplot()

- 分类散点图
- 分类分布图
- 分类估计图

回归图：
lmplot()

Seaborn绘图函数



- 图级函数：通常是一个FacetGrid对象，用来管理图形
- 优点：轻松创建具有多个子图的图形（分面容易）
- 缺点：较难与其他绘图进行组合，与matplotlib.pyplot API的使用不同

- 轴级函数：将数据绘制到单个matplotlib.pyplot.Axes 对象上，该对象是函数的返回值
- 优点：可以按matplotlib.pyplot API的使用方式，添加子图和进行图形设置
- 缺点：分面较难

关系图：seaborn.relplot()

- 可视化变量的统计关系：sns.relplot() [查看API]
- 通过设置kind参数来指定绘制图形的类型，默认绘制的是散点图

- 绘制**散点图**，相当于sns.scatterplot()

```
sns.relplot(data, x, y, kind = 'scatter', hue, col, row, ...)
```

- 绘制**线图**，相当于sns.lineplot()

```
sns.relplot(data, x, y, kind = 'line', hue, col, row, ...)
```

- data：表示数据，通常是长格式DataFrame
- x：表示横轴要绘制的变量；例如DataFrame的某一列
- y：表示纵轴要绘制的变量；例如DataFrame的某一列
- hue：表示颜色分组维度，按照该变量对数据按颜色分组，建立语义映射；例如DataFrame中的一个分类变量
- col：表示将整个图形分成若干列子图（网格）所依据的分组变量
- row：表示将整个图形分成若干行子图（网格）所依据的分组变量

关系图：seaborn.relplot()

sns.relplot()其他常用参数介绍：

- size: 表示元素大小的分组变量；设置之后，不同散点将根据该变量的取值而有不同的大小
- style: 表示元素样式的分组变量；设置之后，不同散点将根据该变量的取值而有不同的样式
- row_order, col_order：重新定义子图排列的行或列的顺序，否则Seaborn将从分类变量的取值自动推断
- palette：调色板，取值可以是字符串、列表或字典；表示在映射 “hue” （色彩语义）时选择要使用的颜色方法
- hue_order: 指定hue的情况下，设置分类变量不同取值下的元素颜色绘制顺序
- size_order: 指定size的情况下，设置分类变量不同取值下的元素大小绘制顺序
- style_order: 指定style的情况下，设置分类变量不同取值下的元素样式绘制顺序
- legend: 图例绘制方式，取值可以为“auto”、 “brief”、 “full” 或 False，默认值为“auto”
- height: 图片高度，默认值是5
- aspect: 图片宽高比，默认值是1，表示宽度等于高度

Seaborn “网格”绘图：FacetGrid()

- sns.relplot()返回的是一个实例化的FacetGrid类 [查看API]
- 使用sns.FacetGrid()可以创建结构化的多子图 “网格”（grid），方便在数据集的不同子集上绘制图表，可以从三个维度（row, col, hue）考察变量之间的关系

```
sns.FacetGrid (data, row, col, hue, col_wrap, height=3, aspect=1, sharex=True, sharey=True, **kwargs)
```

- data：处理后的长格式数据
- row, col：定义划分网格行和列的变量
- hue：定义用颜色分类的变量
- col_wrap：网格列维度的限制
- height, aspect：默认的图片高度和宽高比
- sharex, sharey：网格中的子图是否共用横轴和纵轴

- 创建FacetGrid对象后，可以使用map()方法在每个子图上绘制不同的图形

```
g = sns.FacetGrid(data)
g.map(function, data, x, y, ...)
```

- function：绘图函数，例如sns.scatterplot, sns.lineplot
- data：数据
- x, y：X轴和Y轴要绘制的变量，来自data中的两列

- map_dataframe() 功能类似于map()
- map_dataframe() 适用于使用接受长格式 DataFrame 作为data关键字参数，并在该 DataFrame 中使用字符串变量名称访问数据

Seaborn“网格”绘图：FacetGrid()

- 除了map()方法，FacetGrid还提供了其他方法用于自定义和修饰图形
  - add_legend(): 添加图例
  - set_titles(): 设置每个子图的标题
  - set_xlabels()、set_ylabels(): 设置X轴和Y轴的标签
  - set_xticklabels()、 set_yticklabels()： 设置X轴和Y轴的刻度标签
  - refline(): 设置参照线
  - tick_params(): 修改刻度、刻度标签和网格线
  - savefig(): 保存图像

分布图：sns.displot()

- 可视化变量的分布：sns.displot() [查看API]
  - 除了参数kind取值可以是'hist'（默认值）、'kde'、'ecdf'之外，其他参数的含义和sns.relplot()几乎一致

- 绘制**条形图**或**直方图**，相当于sns.histplot()

```
sns.displot (data, x, y, hue, row, col, kind='hist', **kwargs)
```

- 绘制**核密度图**，相当于sns.kdeplot()

```
sns.displot (data, x, y, hue, row, col, kind='kde', **kwargs)
```

- 绘制**经验累积分布图**，相当于sns.ecdfplot()

```
sns.displot (data, x, y, hue, row, col, kind='ecdf', **kwargs)
```

直方图：sns.histplot()

- 如前所述，使用sns.displot(kind='hist')可以绘制条形图或直方图
- 同时指定kind='hist'和kde=True可以在直方图上再画出拟合的核密度图
  - sns.displot(kind='hist', kde=True)
- 也可以使用sns.histplot() [查看API]

sns.histplot (data, x, y, hue, stat='count', bins='auto', binwidth, binrange, cumulative=False, element='bars', kde=False)

- 纵轴统计量，取值可以是频数、概率、概率密度
- 指定分箱数或分箱方式
- 分箱的宽度
- 分箱边界值
- 是否绘制累积分布
- 图形形状，可以是bars、step、poly
- 改为True可以绘制核密度估计曲线

分类图：sns.catplot()

➢可视化变量的分类：sns.catplot() [查看API]

sns.catplot (data，kind, col, row, order, aspect, height, dodge **kwargs)

- 处理后的长格式数据
- 图像类型
- 在不同行、列绘制子图
- 指定类别变量的绘制顺序
- 控制子图的宽高比和高度
- 将不同分类的图形分开显示

- 使用kind='strip'（或 'swarm'）绘制**分类散点图**，相当于sns.scatterplot()
  - kind='swarm'不能设置jitter，它使用一种算法来调整分类轴上的点，使其不重合，也被称为“蜂群图”，可以用sns.swarmplot()绘制

sns.catplot (data, x, y, hue, row, col, kind='strip', jitter, **kwargs)

jitter参数可以控制抖动的幅度，或者禁用抖动；默认为True

- 使用kind='box' 或 'violin', 绘制**分类分布图**，相当于箱型图sns.boxplot()或小提琴图sns.violinplot()

sns.catplot (data, x, y, hue, row, col, kind='box', **kwargs)

- 使用kind='bar'、'point' 或'count'绘制**类别估计图**（带误差估计），相当于条形图sns.barplot()、点估计图sns.pointplot()、分类变量条形图sns.countplot()

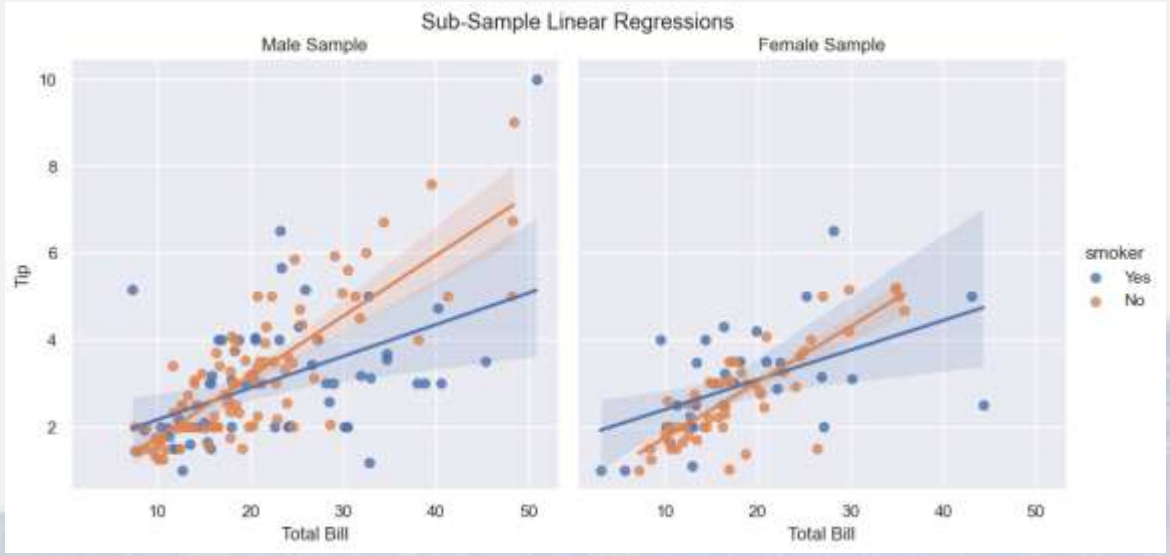
sns.catplot (data, x, y, hue, row, col, kind='bar', **kwargs)

回归图：sns.lmplot()

- 可视化变量的回归模型：sns.lmplot() [查看API]
- 使用sns.lmplot()不仅能绘制回归线，还能将回归结果按不同的分类变量进行分面（faceting）显示，适用于探索变量间的关系。回归图也可以使用sns.regplot()

sns.lmplot (data, fit_reg, ci, n_boot, line_kws, scatter_kws, lowess, order, logistic, **kwargs)

- 是否显示回归线，默认True
- 设置置信区间，默认95% CI
- 控制bootstrap次数，默认1000次
- 自定义回归线和散点的样式
- 拟合局部加权回归曲线
- 设置回归模型的多项式阶数
- 当因变量是二分类时，绘制逻辑回归曲线

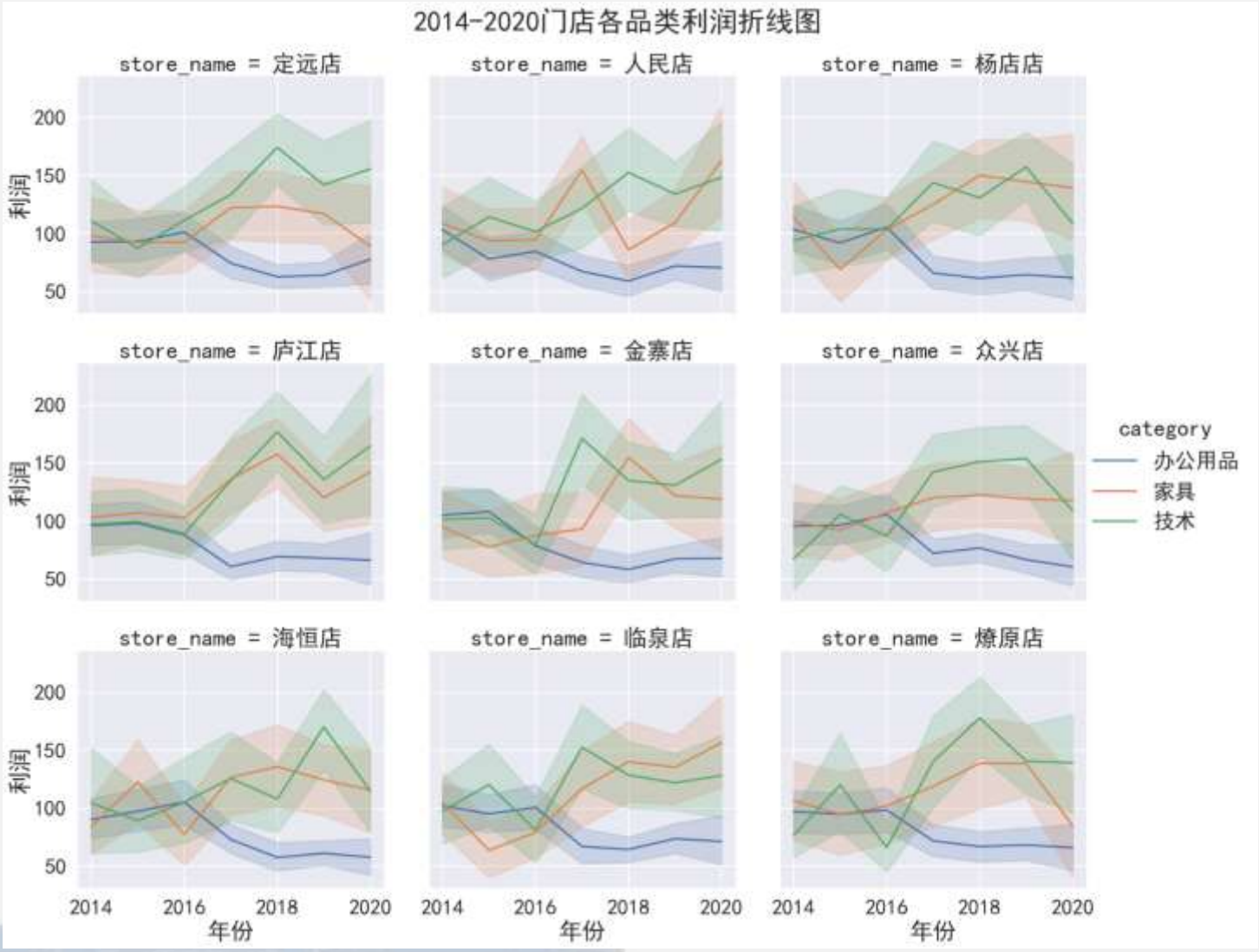


对Smoker和Non-Smoker两个群体分别在不同性别的样本中进行回归

Seaborn课堂练习

根据提供的数据集orders.txt

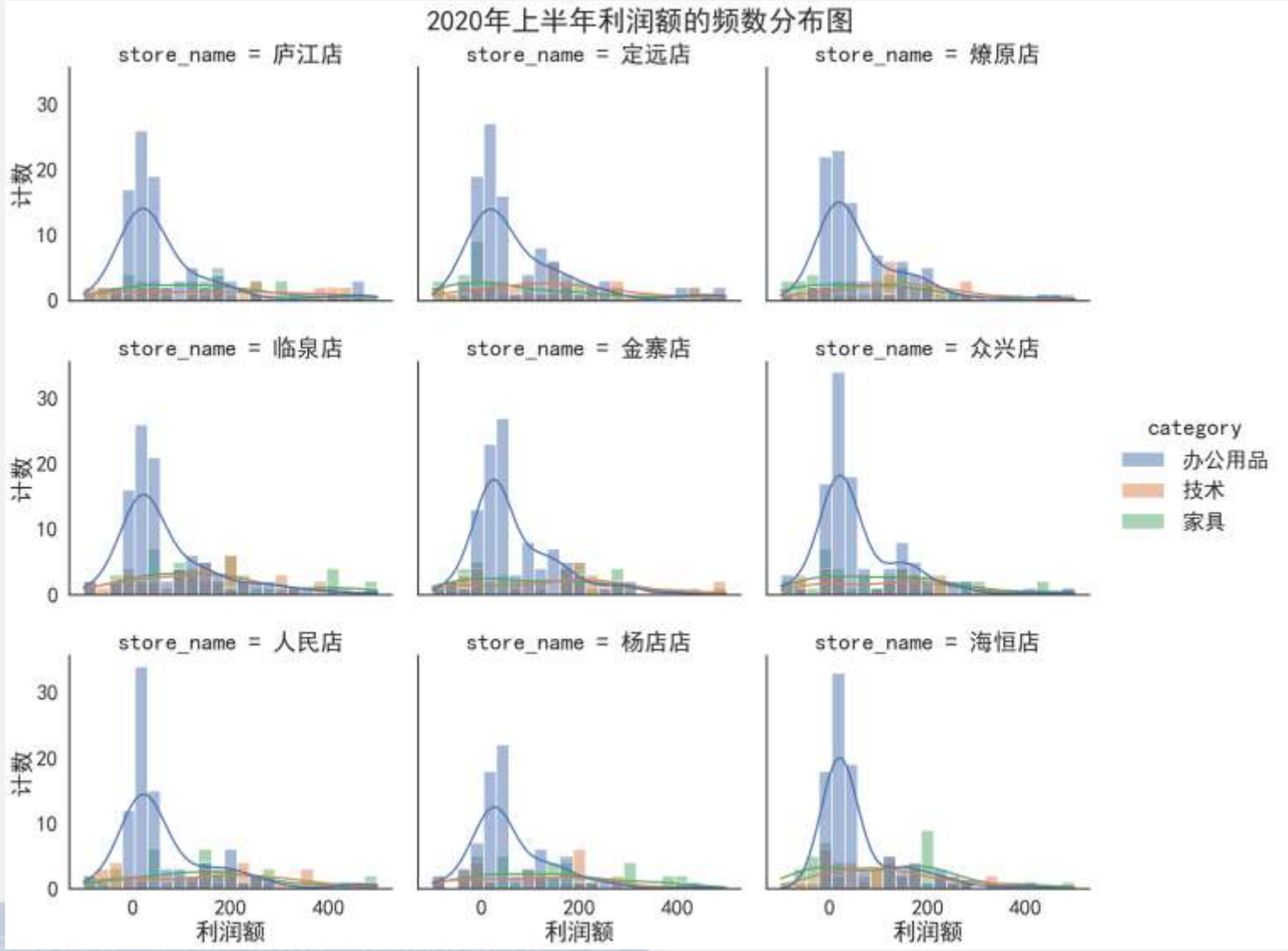
1. 使用sns.relplot，绘制各个门店2014-2020期间，不同产品品类(category)的利润折线图



Seaborn课堂练习

根据提供的数据集orders.txt

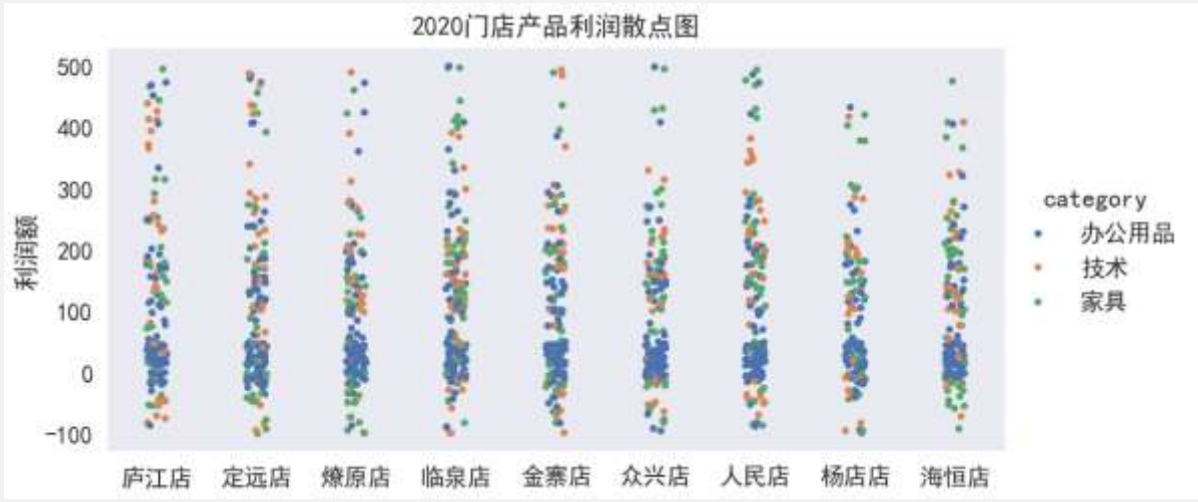
2. 使用sns.displot，绘制2020年各个门店不同产品品类的利润直方图和核密度曲线

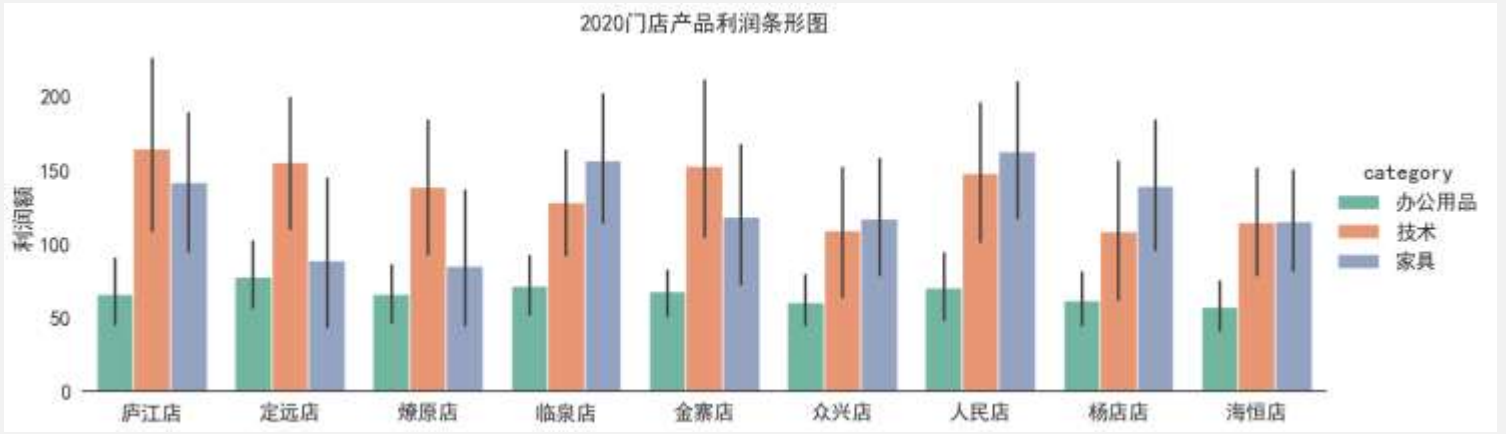


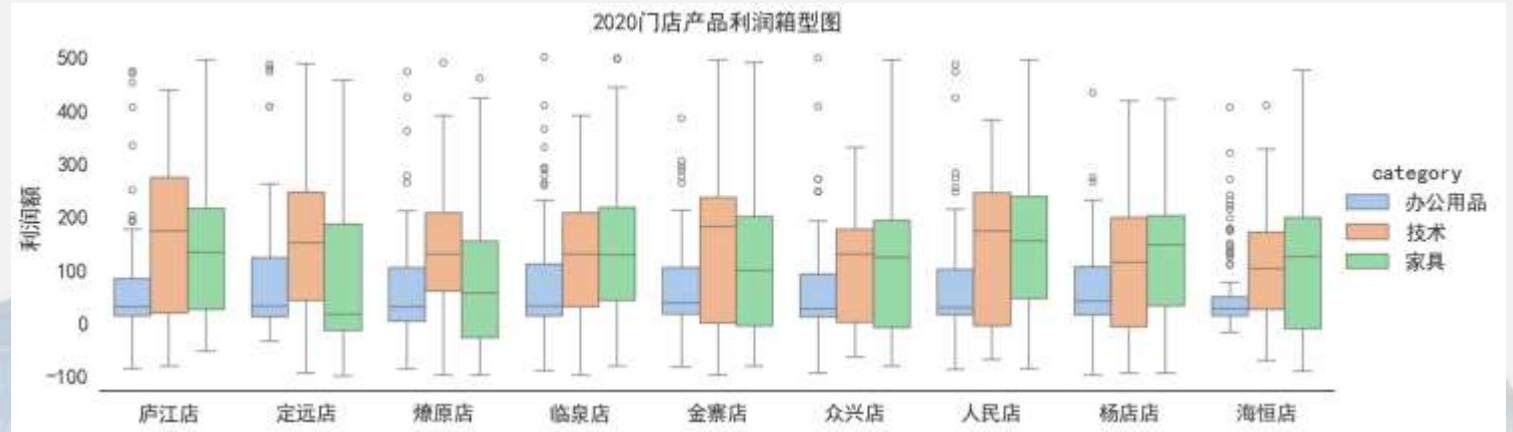
Seaborn课堂练习

根据提供的数据集orders.txt

3. 使用sns.catplot，绘制2020年各个门店不同产品品类的利润散点图、箱型图、条形图



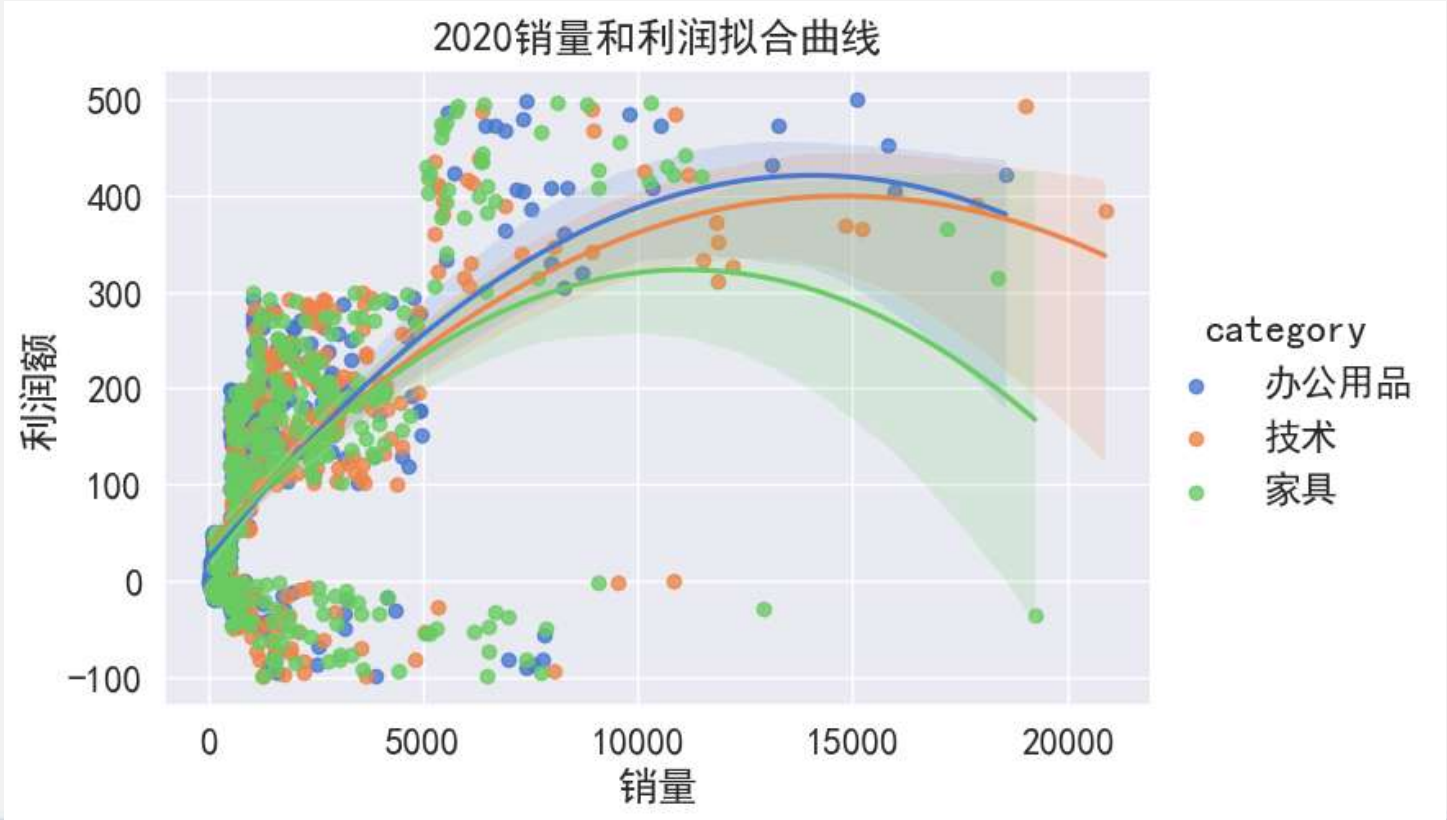




Seaborn课堂练习

根据提供的数据集orders.txt

4. 使用sns.lmplot，绘制2020年各个产品类别的销量和利润关系的拟合曲线



Seaborn绘图函数：sns.pairplot()

- 可视化成对变量的关系：sns.pairplot() [查看API]
  - pairplot()绘制数据集中所有两两组合的数值变量（即成对的变量），展示这些变量对之间的关系。主对角线上绘制的是每个变量的分布（默认是'auto'），其余部分显示的是各变量对之间的散点图

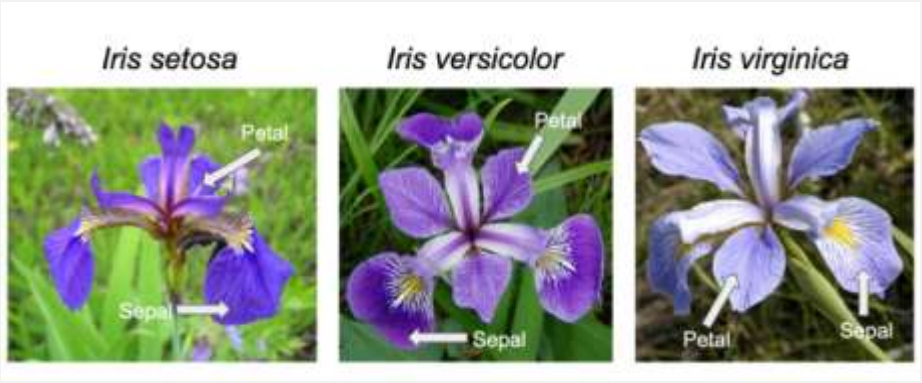
sns.pairplot (data, hue, vars, x_vars, y_vars, kind, diag_kind, aspect, height, plot_kws, diag_kws, **kwargs)

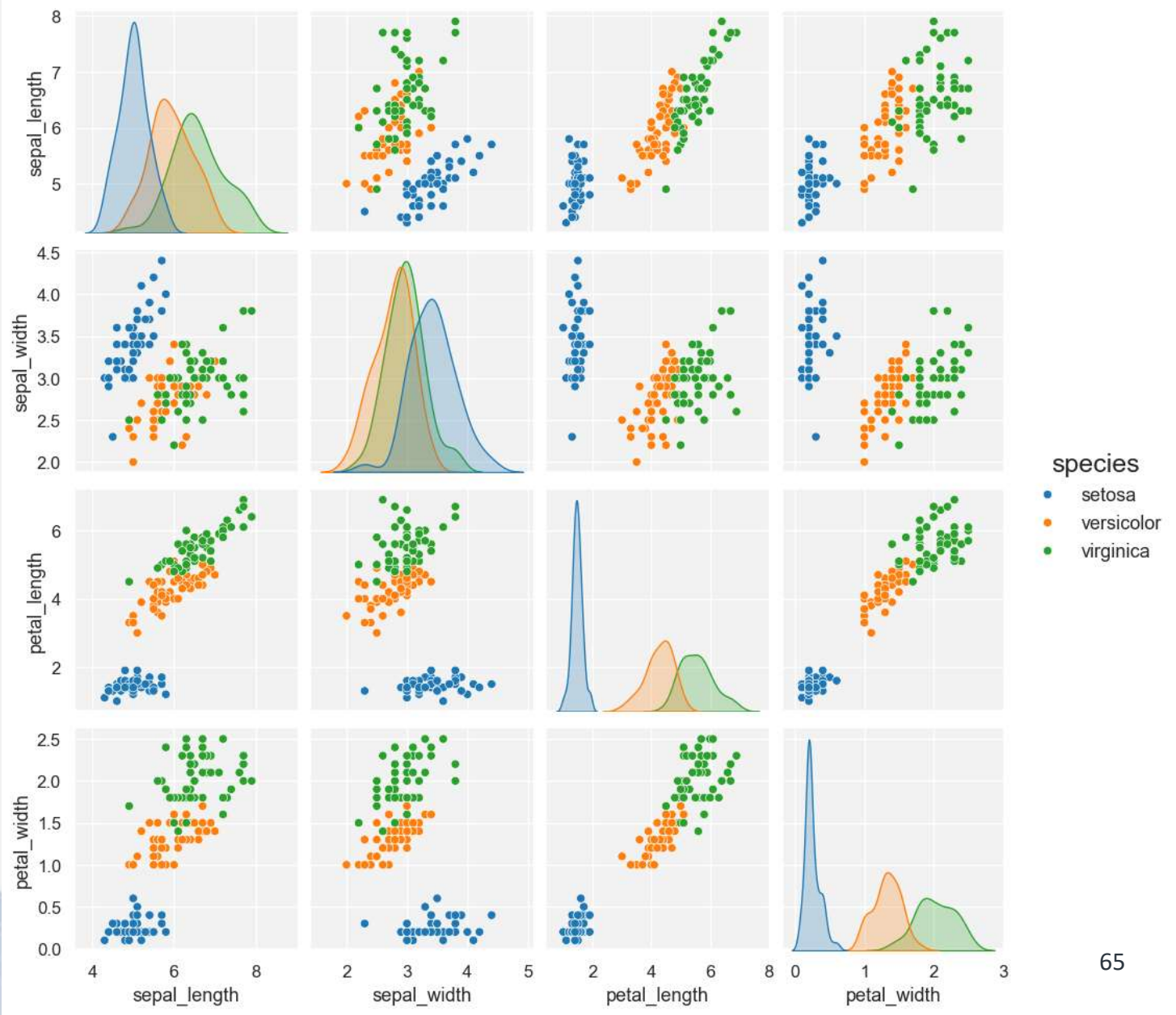
- data, hue: 根据分类变量对不同类别的数据进行区分
- vars, x_vars, y_vars: 指定要可视化的特定变量子集，x和y分别指定横轴和纵轴上要显示的变量
- kind, diag_kind: 非对角和对角线的绘图方式
- aspect, height: 控制生成图形的高度和宽高比例
- plot_kws, diag_kws, **kwargs: 自定义非对角线（即散点图）和对角线（如直方图、核密度图）部分的样式

  - 使用sns.pairplot()函数后，返回的是一个PairGrid对象 [查看API]
  - PairGrid对象代表了一个由多个子图组成的网格布局，并且是为成对变量绘制而专门设计的网格

Seaborn绘图函数： sns.pairplot()

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

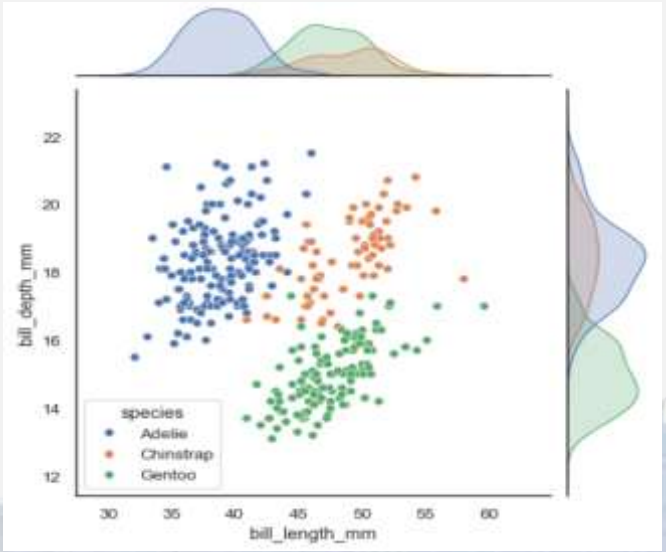




Seaborn绘图函数：sns.jointplot()

- 可视化成对变量的关系：sns.jointplot() [查看API]
  - sns.jointplot()将散点图、回归线或六边形图与单变量分布图（如直方图或核密度图）结合在一起，通过双变量和单变量的图形表现出变量间的联合分布及其边缘分布情况
  - 在只有两个数值变量关系时，jointplot()与pairplot()基本类似，只在绘图类型上有差异

sns.jointplot (data, x, y, hue, kind, joint_kws, marginal_kws,**kwargs)



kind: *“scatter” | “kde” | “hist” | “hex” | “reg” | “resid”*

- 内部显示不同类别下，x和y的散点图，上侧和右侧显示不同类别下，x和y各自的核密度曲线
- 使用 sns.jointplot() 函数后，返回的是一个JointGrid对象 [查看API]

Seaborn绘图函数（轴级）

函数	核心参数说明	图表类型
scatterplot()	x, y, hue, size,style(分类显示不同形状), palette, sizes, sizes_order, markers,style_order,legend	散点图
lineplot()	x, y, units(单位均值线), weights(权重), sizes(线宽), dashes(虚实线),n_boot(置信区间迭代次数), err_style(置信区间样式), err_kws(置信区间其他参数)	线图
histplot()	x, y, stat(纵轴统计量), binwidth(直方图宽度), binrange(横轴范围), cumulative(转为累积直方图), common_norm(数据归一化), multiple(多类别叠加方式), element(样式), kde(核密度曲线), kde_kws(核密度样式), line_kws(线图样式)	直方图
kdeplot()	data1, data2, cumulative(核密度累积分布), bw_method(核密度带宽), levels(二维等高线层数), , shade_lowest(核密度中最低范围着色), n_levels(二维等高线层级数量)	核密度图
ecdfplot()	x, y, stat(分布统计量), complementary(绘制累积分布函数), log_scale(轴刻度对数显示)	经验累积分布图
rugplot()	x, y, height(观测条高度), axis(观测条所在轴), color(观测条颜色)	轴分布图（“地毯图”）
stripplot() swarmplot()	x, y, hue, jitter(重叠间距), dodge(分离数据条), orient(条形图方向), size(标记大小), edgecolor(点周线条颜色), linewidth(边框线宽)	分类散点图 （非重叠）分类散点图

Seaborn绘图函数（轴级）

函数	核心参数说明	图表类型
boxplot()	x,y, hue, saturation(颜色饱和度), width(元素宽度), linewidth(边框线宽), fill(是否填充箱子), fliersize(异常值标记大小)，whis(箱须长度，默认为1.5IQR), log_scale(设置对数刻度)	箱型图
violinplot()	x,y,hue, inner(内部图), split(分裂显示), bw_method(核密度带宽), density_norm(密度归一化), scale(小提琴大小比例), bw(核密度方式)	小提琴图
boxenplot()	x,y,hue,width_method(盒形宽度的表示方式), k_depth(盒形图数目), outlier_prop(异常值数据比例), trust_alpha(信任区间显著水平)	增强型箱线图
pointplot()	x, y, hue, estimator(集中趋势估计), errorbar(误差条显示方式), n_boot(迭代次数), linestyles(线形), err_kws(误差条参数), ci(置信区间范围), errwidth(误差线厚度), join(连接数据点)	带误差线的点估计图
barplot()	x, y, hue, estimator(集中趋势估计), errorbar, n_boot(置信区间迭代次数), errcolor(线条颜色)	带误差线的条形图
countplot()	x, y, hue, orient(绘图方向), stat(统计方式，可以为‘count’, ‘percent’, ‘proportion’, ‘probability’), dodge(使用hue映射时，图形是否变窄并移动以消除重叠)	分类变量条形图
regplot()	x, y, data, x_estimator(x轴的集中趋势估计), x_bins(x轴区间数), x_ci(x轴置信区间)，scatter(加入散点图), fit_reg(加入回归线), ci(整体置信区间), seed(设置种子), order (回归阶数), logistic(使用逻辑回归), lowess(使用局部加权回归), logx(对x进行对数转换), robust(使用稳健回归), truncate(消除极端值影响)	线性回归拟合图
residplot()	x, y, data,lowess(使用局部加权回归), order(多项式阶数), robust(使用稳健回归)	线性回归残差图

本章小结

- Python数据处理
  - NumPy
  - Pandas
- Python可视化库
  - Matplotlib
  - Seaborn
    - 图级函数：relplot, displot, catplot, lmplot, pairplot, jointplot
    - 轴级函数：见表格整理

本章完

